

XL

KENSHO PROJECT RETROSPECTIVE

Excel Lite

What 188 agent jobs built, where parallelism worked, why integration still converged on Kai, and what must change before a trusted public release.

148

WORKSHEET FUNCTIONS

1,585

CONFORMANCE CASES

188

KENSHO JOBS

1.57

EFFECTIVE CONCURRENCY

LOCAL DEMO READY WITH DOCUMENTED CAVEATS

PUBLIC macOS V1 NOT READY

Analysis date: 10 July 2026

Audited revision: 4f112c930593

Local-first product study / no cloud dependency

Contents

A product verdict and an organizational retrospective, backed by local acceptance evidence and Kensho's recorded execution data.

Project Overview, Motivation, And Requirements	3
Verdict	3
Product Evidence	4
Evidence Summary	5
Scorecard	6
What Worked	7
What Did Not Work	8
Topology Findings	9
Product Launch Gaps	10
Hypotheses For Better Bootstrapping	10
Recommended Starter Topology	11
Framework Bug Ledger	12
Promotion Gates	12
Existing Supporting Evidence	13
Audit Constraints	13
Final Assessment	13

Project Overview, Motivation, And Requirements

Excel Lite was both a product build and a stress test of Kensho's autonomous engineering model. The product had to be immediately legible to a human: type a formula, edit its precedents, save the workbook, reopen it, and see the right result. At the same time, its 148-function calculation surface created a large body of work that could be split across independent agents behind a stable evaluator contract.

That combination made the project more revealing than a conventional feature demo. A plausible spreadsheet shell is easy to fake; a calculation engine with broad conformance, dependency-aware recalculation, native persistence, history, import/export, and a packaged desktop application is not. The project could therefore test two questions at once:

- 1 Can Kensho coordinate a large parallel fan-out without losing semantic consistency?
- 2 Can it bring the integrated product through the less parallel release tail: native workflows, safety, evidence, packaging, and distribution trust?

The intended product was a fully local macOS spreadsheet with:

- A headless Rust engine implementing 148 documented worksheet functions.
- Automatic dependency-aware recalculation, range semantics, cycle handling, and Excel-style errors.
- A responsive single-sheet grid with 1,048,576 rows, 16,384 columns, formula entry, selection, formatting, and structural edits.
- Lossless native workbook files, CSV interchange, best-effort XLSX import, and undo/redo.
- A Tauri desktop shell that remains offline and does not require an account, API key, hosted service, or cloud provider.
- Objective conformance, coverage, native workflow, packaging, signing, and notarization gates.

The human intent was ambitious but simple: exploit safe parallelism aggressively, preserve quality through verification and review, let Kai manage the organization, and surface only real project-level decisions. Excel Lite therefore evaluates Kensho as much as it evaluates the spreadsheet.

Verdict

LOCAL DEMONSTRATION	PUBLIC macOS V1	KENSHO AUTONOMY
READY	NOT READY	PARTIAL

Kensho produced a real, useful local spreadsheet application with an unusually strong calculation core. Current `main` passes 1,585 conformance cases, 1,341 Rust tests, 38 frontend tests, and a 96.29% function/evaluator coverage gate. A fresh Tauri build creates a working arm64 application and DMG; the packaged app launches, recalculates formulas, opens native dialogs, and passes offline scans.

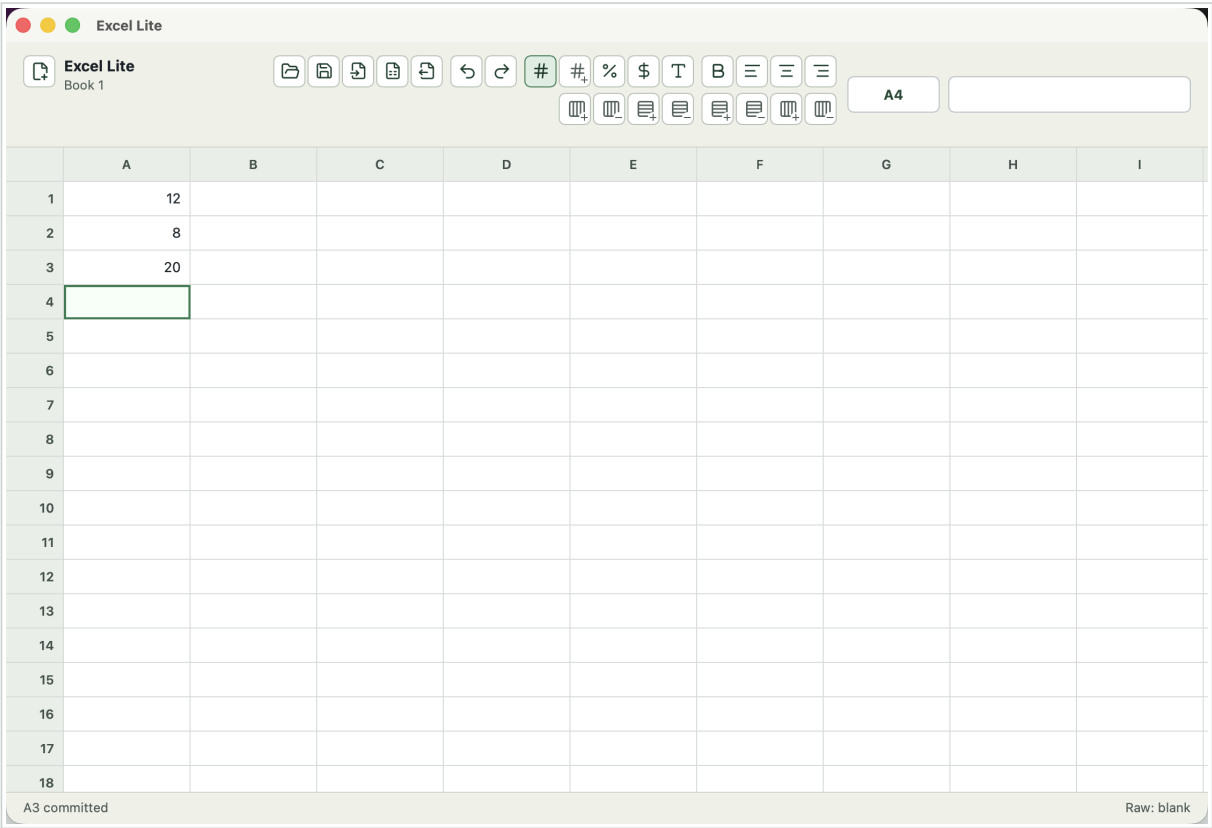
The product is ready for a controlled local demonstration. It is not ready to be presented as a public macOS v1. The lower half of the million-row grid is unreachable, destructive actions can discard unsaved work without warning, the native UI lacks an automated end-to-end lane, and the package is neither Developer ID signed nor notarized. The oracle, security, identity, and release-documentation claims are also incomplete.

As a Kensho experiment, Excel Lite proved that broad implementation fan-out can work: 188 jobs were recorded, 182 completed, and the function epic alone contained 147 jobs. It did not prove hands-off self-sustaining autonomy. Effective concurrency was 1.57 with a peak of 4; almost all jobs ran as direct worker deliveries; the declared verifier/reviewer pipeline was unused; and Kai became the central merge, validation, repair, and progress-reconciliation point. External overseer prompts repeatedly restored parallelism or corrected stale state.

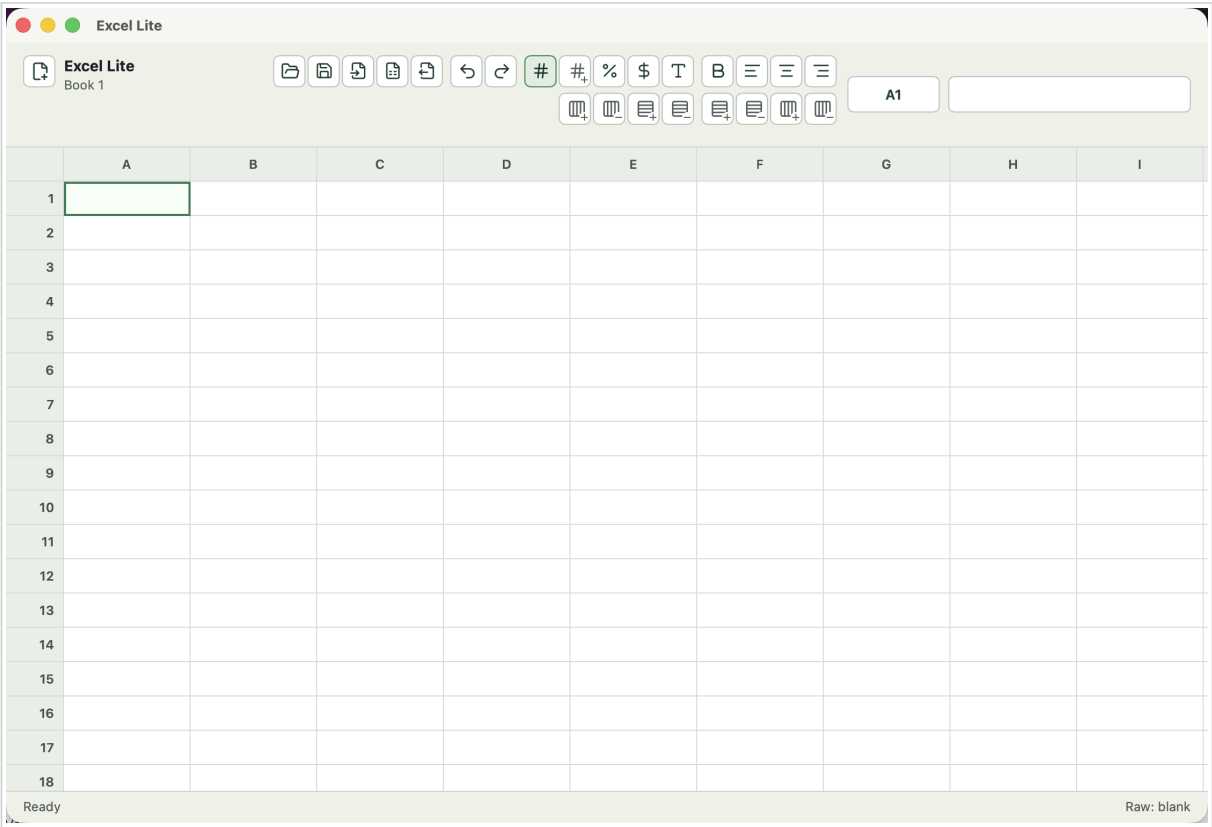
The central lesson is not that Kensho needs more agents. It needs better delegation of integration authority, mandatory machine verification, independently exercised review lanes, and control-plane state that reconciles itself against the repository before asking for more work.

Product Evidence

The native application was exercised directly before and after packaging; these are tracked verification captures, not design mockups.



Native recalculation workflow



Fresh packaged application boot

Evidence Summary

Product commands were run against the audited revision:

Area	Command or probe	Result
Conformance corpus	<code>python3 scripts/validate_conformance_cases.py</code>	1,585 cases across 152 files passed.
Function completeness	<code>python3 scripts/audit_conformance_coverage.py</code>	148/148 registered functions; no missing, duplicate, or orphan implementations; every dedicated file met the five-case count.
CI contracts	<code>python3 scripts/validate_ci_contracts.py</code>	Passed.
Rust formatting and lint	<code>cargo fmt --check</code> and repository CI-equivalent <code>cargo clippy</code>	Passed.
Rust tests	<code>cargo test --workspace</code>	1,341 tests passed.
Core coverage	<code>cargo llvm-cov</code> plus repository gate	96.29% aggregate function/evaluator coverage against a 90% gate.
Frontend checks	<code>npm run check</code> , <code>npm test</code> , <code>npm run build</code>	0 check errors/warnings; 38 tests across 3 files; production build passed.
Offline verification	<code>npm run verify:offline</code> before and after packaging	Passed. No runtime network dependency was found.
Native package	<code>npm run tauri:build</code>	Produced <code>Excel Lite.app</code> and an arm64 DMG.
DMG integrity	<code>hdiutil verify</code>	Passed; image contains the app and Applications link.
Packaged launch	Fresh launch plus native accessibility/status probes	Passed at 1120x760; app reported <code>Ready</code> ; formula recalculation and native open dialog were exercised.
Responsive layout	Live browser checks at 1120x760 and 880x560	No clipping, overlap, page overflow, or console errors.
Bottom-of-sheet probe	Scroll/navigation against the live grid	Failed: CSS height clamps at 16,777,216 px, so the final viewport begins around row 524,273.
Distribution trust	<code>codesign --verify --deep --strict</code> and <code>spctl --assess --type execute</code>	Failed; package is ad-hoc signed with no Team ID and is not notarized.

Checks that remain missing or incomplete:

Missing check	Impact
Automated packaged-app workflow suite	Manual native evidence does not protect formula entry, save/reopen, import, history, dialogs, or boundary navigation on every change.
Million-row performance benchmark	The specified final row is unreachable and the 60 fps target is not demonstrated.
Reproducible LibreOffice oracle	<code>scripts/regen_oracle.py</code> is a non-writing scaffold; committed expectations are not independently regenerated in automation.
Clean-machine signing/notarization test	The artifact cannot yet be trusted as a normal public macOS download.
Automated accessibility audit	Keyboard and screen-reader behavior are not comprehensively measured.

Kensho process evidence came from the daemon event stream, job ledger, outcome report, manager state, topology, and ten recorded feedback items:

Metric	Value
Jobs	188
Done / failed	182 / 6
Worker / reviewer / verifier jobs	187 / 1 / 0
Product-job runtime sum	About 21h51m
Job-ledger window	About 43h27m

Metric	Value
Effective / peak concurrency	1.57 / 4
Recorded input / output / reasoning tokens	548.8M / 3.84M / 1.35M
Worker cached input	526.0M of 545.4M worker input
Function-fanout jobs	147
Pipeline jobs / PR-backed jobs	0 / 0
Daemon events	840
Dispatch / exit / manager-idle-wake events	194 / 247 / 24
Authority-violation events	36, under audit mode
Concurrency-ceiling adjustments	76

Scorecard

Delivery

Dimension	Score	Evidence
Calculation core	Strong pass	148 functions, 1,585 conformance cases, 1,341 Rust tests, 96.29% measured coverage.
Recalculation and errors	Pass for tested surface	Direct, cascade, diamond, fan-out, isolation, range, cycle, and error behavior have dedicated tests.
Persistence and history	Pass for scoped behavior	Native round-trip includes a 1,000-case property test; save/load, CSV, best-effort XLSX import, and undo/redo paths exist.
Native application	Partial/pass	Packaged app builds and launches; calculation and file-dialog workflows have direct evidence. Automated native e2e is absent.
Grid scale	Fail against v1 spec	Final row is unreachable because browser layout height clamps before 1,048,576 rows.
Document safety	Fail	No dirty-state model or confirmation before New, Open, close, quit, or other destructive transitions.
UI contract	Partial	Core spreadsheet shell works, but range drag selection, range clear, date-format action, and specified interaction details remain incomplete.
Local-only constraint	Pass	Runtime is offline, locally packaged, and free of cloud/account dependencies.
Distribution	Fail for public release	Ad-hoc signature, no notarization, incomplete icons/identity/docs, no tagged release workflow.
Release evidence	Partial	Core evidence is strong; native automation, oracle provenance, accessibility, security hardening, and clean release provenance are incomplete.

Process Efficiency

Excel Lite used far more parallelism than the earlier chess project, but much less than its job count suggests. Peak concurrency reached 4 and the function library was decomposed into 147 narrow jobs. Effective concurrency of 1.57 means the organization still spent substantial time with one active delivery or waiting on integration, host load, manager review, or state reconciliation.

The cost profile was also uneven:

Signal	Interpretation
188 total jobs	Kensho handled a genuinely large work graph rather than a few coarse prompts.
147 function jobs	The stable evaluator contract supported broad, low-conflict fan-out.
187 worker jobs, 1 reviewer, 0 verifier	Role capacity did not become an exercised quality pipeline. Kai performed most integration and acceptance work.

Signal	Interpretation
6 failed job records	Failures were recoverable, but four early walking-skeleton failures and two function defects required replacement work.
548.8M input tokens	Expensive in nominal context volume, though 526.0M worker tokens were cached. Repeated whole-repo context and full gates still deserve optimization.
21h51m runtime sum over a 43h27m ledger window	Available parallelism was not continuously converted into useful execution.
76 scheduler ceiling changes	Adaptive host-load protection materially influenced dispatch; at one point the ceiling fell to zero.
24 manager idle wakes	The daemon frequently had to re-engage the manager rather than progressing through an explicit autonomous queue.

The largest individual jobs were not random: the final UI product pass used 17.6M input tokens; structural commands used 11.3M; XLSX import used 7.4M; and complex formula/reference contracts occupied several of the other top slots. Those are integration-heavy boundaries. Once the leaf functions existed, the bottleneck moved from implementation supply to contract integration and product-level verification.

The Adversarial Gate

The gate caught real defects, but it was not the independent multi-role system described by the topology.

Job	Finding	Outcome
xl-fn-db	Iterated up to unbounded finite <code>period/life</code> , allowing pathological hangs.	Original rejected; bounded replacement implemented and accepted.
xl-fn-timevalue	Accepted arbitrary text prefixes instead of the intended time grammar.	Original rejected; corrected replacement implemented and accepted.
Four Slice 1 jobs	Initial UI/engine/recalc/acceptance attempts failed or were superseded during walking-skeleton integration.	Replacement integration jobs established the working baseline.

This is meaningful quality evidence: DB and TIMEVALUE would have been user-visible semantic or availability defects. The problem is organizational. Only one job was assigned to the reviewer role, no job went through the verifier role, and none used the declared four-step `ticket_to_pr` pipeline. Most accepted branches were reviewed, merged, gated, and manually closed by Kai. The review function existed, but reviewer independence and pipeline-level reproducibility did not.

No verified evidence shows these rejected defects escaped into the final product. The more important escaped-defect class was at the product boundary: the calculation engine was extensively tested while grid reachability, unsaved-work safety, and distribution trust remained incomplete until the final readiness audit.

What Worked

- 1 The evaluator contract enabled real fan-out. Narrow function branches could be implemented in parallel with limited merge overlap, and the registry/coverage audits prevented missing or orphaned work.
- 2 Objective gates made the core credible. The report does not rely on screenshots alone: exact conformance counts, coverage, property tests, CI-contract checks, and package integrity can all be reproduced locally.
- 3 Review found defects outside happy-path examples. DB's unbounded work and TIMEVALUE's permissive parsing are precisely the kinds of mistakes a shallow function-count audit would miss.
- 4 Local-first architecture held. The application works without accounts, API keys, hosted services, telemetry, or a cloud provider. Packaging and offline scans reinforce that claim.
- 5 The team recovered from a weak walking skeleton. Four failed early jobs did not poison the architecture; replacement integration work established a stable engine/UI contract that supported later fan-out.
- 6 Kai correctly challenged stale supervisor state. When an external prompt claimed only 87 of 148 functions were complete, Kai checked `main`, found all 148 registered and covered, and declined to dispatch duplicate work.
- 7 The release audit remained honest. A working application was not promoted to public-v1 status when signing, notarization, document safety, grid scale, and native automation were still missing.

- 8 Feedback captured reusable framework failures. Cleanup blocks, direct-job bounce limits, stale progress, local-branch handoff errors, and undeclared external-supervisor communication were recorded as durable Kensho feedback rather than treated as one-off operator frustration.

What Did Not Work

- 1 Kai became a serialized integration manager. It repeatedly inspected branches, ran aggregate gates, merged, closed jobs, acknowledged daemon messages, and cleaned worktrees. That protected quality but limited throughput and autonomy.
- 2 The declared pipeline was not the actual process. The topology defined implement, verify, review, and manual approval steps, yet all 188 ledger entries were direct jobs and no verifier job ran. Configuration cannot be credited as a control if execution bypasses it.
- 3 Review capacity was largely idle. Three reviewer replicas were configured, but one reviewer job was recorded. Most adversarial reasoning happened inside the manager context, weakening independence and increasing context pressure on Kai.
- 4 The control plane generated stale work. Supervisor prompts twice reported 87/148 functions after repository audits proved 148/148. Numerous manager-action and idle-wake signals referred to already merged or closed work.
- 5 Parallelism needed external prompting. Overseer messages repeatedly asked Kai to refill capacity or inspect idle state. The organization did not consistently derive the next safe wave from desired state on its own.
- 6 Adaptive scheduling could stall the queue. Host-load logic temporarily reduced the concurrency ceiling to zero. Protection against overload is necessary, but a zero ceiling with no escalation or slow lane makes a healthy daemon look dead.
- 7 Cleanup was brittle. Completed worktrees were repeatedly held open by exited Computer Use helper processes; other cleanup attempts failed on `lsOf` exit behavior. These were transient, but they consumed manager attention.
- 8 Direct local-branch delivery had false negatives. Already merged branches could be reported as missing deliverables or failed because their diff against `main` was empty after merge. Kai manually reconciled several records.
- 9 Function completion obscured the product tail. The system optimized the countable 148-function fan-out while release-critical cross-cutting work such as dirty state, bottom-of-grid navigation, native e2e, signing, and notarization remained late or absent.
- 10 Visual and native verification remained partly artisanal. Screenshots and accessibility probes were valuable, but they were not a committed deterministic test lane and could not prevent regression automatically.

Topology Findings

Used vs. Idle

Used heavily:

- **worker**: 187 jobs, including 147 function-fanout jobs and later persistence, structural, UI, and packaging work.
- **manager** / Kai: persistent coordination, branch review, merges, aggregate gates, job reconciliation, cleanup, and operator reporting.

Used lightly or not at all:

- **reviewer**: one recorded job despite three configured replicas.
- **verifier**: zero recorded jobs despite two configured replicas.
- **ticket_to_pr**: declared with implement, verify, review, and manager approval, but no ledger job used it.
- PR delivery: intentionally absent in the local-only project, but no pipeline-native local branch equivalent replaced it.

Binding Bottleneck

Worker supply was not the final bottleneck. The binding constraint was serialized integration authority:

- Kai was the only reliable entity that understood product state, branch state, stale daemon state, and release gates together.
- Every leaf could be parallel, but registry integration, contract changes, UI wiring, and acceptance evidence converged on shared files and one manager context.
- The system lacked category-level integrators who could own coherent subsets such as financial functions, date/time semantics, persistence/history, or native product acceptance.
- Machine verification was not an enforced pipeline stage, so Kai repeatedly reran and interpreted gates before accepting branches.

This is a recursive delegation problem. When global management became too complex, Kensho needed to create bounded units with their own integration managers, verifier capacity, and escalation contracts instead of adding more undifferentiated workers.

Static Configuration Cost

The topology looked richer than the run it produced: four worker replicas, two verifiers, three reviewers, a persistent manager, and a four-step pipeline. Because direct jobs bypassed the pipeline, the effective organization was closer to four workers reporting to one hands-on manager.

Concrete costs:

- Idle specialized capacity while Kai absorbed verification and review.
- No uniform evidence bundle for 188 jobs.
- Manual correction of local-branch delivery and already-merged states.
- Large manager context carrying details that should have lived in unit-level state.
- Work queues driven partly by external prompts instead of reconciled desired state.

Missing Capabilities

- Pipeline-native local branch delivery, including bounce/retry on the rejected commit.
- A deterministic verifier that emits machine-readable evidence before LLM review.
- Category or epic managers with bounded authority, independent queues, and explicit merge surfaces.
- Repository-derived progress reconciliation, so counts come from audits and the job graph rather than stale prose.
- Role-aware deliverable validation after a branch has already merged.
- A native visual QA lane that can launch, interact, capture, and compare packaged applications.

- Scheduler liveness rules that preserve at least a low-rate lane or escalate sustained zero capacity.
- Release-state modeling that separates demo-ready, feature-complete, release-candidate, and distributable states.

Product Launch Gaps

The product-tail findings are summarized here so the retrospective remains useful as a release decision.

P0 Blockers

Blocker	Current behavior	Required resolution
Million-row grid	CSS height clamps at 16,777,216 px; the final viewport begins around row 524,273.	Segmented/rebased virtualization with final-row and final-column interaction tests plus performance evidence.
Unsaved-work safety	New, Open, close, and quit have no durable dirty-state warning.	Dirty-state model, current-path tracking, Save/Save As semantics, destructive-transition confirmation, and native e2e coverage.
macOS distribution trust	App is ad-hoc signed; strict verification and Gatekeeper assessment fail; notarization/stapling are absent.	Developer ID signing, notarization, stapling, quarantined clean-machine assessment, and final artifact smoke.

P1 Evidence And Product Gaps

Gap	Required resolution
Native CI and e2e	Build and exercise the packaged app automatically; retain logs and screenshots on failure.
UI contract completion	Resolve drag range selection, selected-range clear, date formatting, command-contract drift, accessibility, and virtualization benchmarks.
Reference oracle	Implement a pinned LibreOffice adapter and machine-reconcile regenerated expectations, deviations, and semantic case quotas.
Security posture	Enable a tested restrictive CSP; automate npm/Rust advisory policy; publish an SBOM or dependency inventory.
Public identity and docs	Finalize bundle ID, icons, architecture matrix, license, security/support policy, screenshots, limitations, changelog, and tagged release provenance.

Hypotheses For Better Bootstrapping

These are testable hypotheses for the next broad, locally verifiable application.

- 1 Recursive units outperform a flat worker pool once one manager owns more than one integration domain.** Test a global Kai with bounded function, persistence, UI, and release units. Each unit receives a contract, owns its queue, and escalates only cross-unit decisions. Measure manager interventions, stale-state corrections, merge conflicts, and effective concurrency.
- 2 Wave-level verification is more efficient than full-suite repetition on every leaf.** Run narrow per-function tests on leaf jobs, then execute registry, conformance, coverage, and whole-workspace gates once per coherent wave. Measure runtime, cached input, defect escape, and time to feedback.
- 3 A mandatory machine verifier increases reviewer independence.** Require every delivery to produce a structured evidence record before review. Reviewers should inspect semantic gaps and diff risk rather than reconstructing command output. Measure reviewer utilization, review tokens, false accepts, and false rejects.
- 4 Progress should be computed, not narrated.** Derive function counts, branch state, gate state, and outstanding requirements from repository and daemon records. Treat prose status as commentary only. Recreate the stale 87/148 scenario and verify that duplicate dispatch is impossible.
- 5 Integration and product-tail tracks should start before leaf fan-out finishes.** Keep function waves running while separate units continuously exercise the packaged app, document lifecycle, boundary navigation, accessibility, and release evidence. Measure how many P0/P1 findings appear only after feature completion.

- 6 **Low-risk work can auto-merge under policy while cross-cutting work remains gated.** Auto-accept a narrow function only when scope, focused tests, wave verifier, and independent review pass. Keep manual approval for evaluator contracts, file formats, security, UI architecture, and release claims. Measure cycle time and escaped defects.
- 7 **Scheduler liveness needs an explicit degradation mode.** When host pressure persists, retain one low-rate job or emit a blocking escalation with evidence instead of silently holding a zero ceiling. Measure time spent at zero useful concurrency and unnecessary restart/nudge events.

Recommended Starter Topology

The next Excel-scale project should start with responsibility-shaped capacity rather than a single large pool:

Role or unit	Initial shape	Responsibility
Kai / global manager	1 persistent	Own spec, desired state, cross-unit contracts, promotion gates, and operator communication. Do not perform routine leaf merges.
Domain unit managers	2-4 bounded units	Own a coherent subsystem, queue, integration branch, wave gates, and escalation boundary. Scale only when independent domains exist.
Workers	2 per active unit, elastic	Implement narrow slices with focused tests and explicit deliverables.
Deterministic verifiers	2 shared	Run focused gates per leaf and aggregate gates per wave; publish structured evidence.
Adversarial reviewers	2 shared	Review semantics, boundary cases, architecture, and claims after verifier success.
Product verifier	1 persistent or event-driven	Exercise the integrated local application, including screenshots, native workflows, performance boundaries, and offline behavior.
Release unit	Event-driven	Own identity, docs, provenance, signing, notarization, checksums, and distributable-artifact truth.

Recommended flow:

- 1 Kai converts the spec into a machine-readable requirement graph and assigns domains.
- 2 Unit managers dispatch narrow leaves up to their conflict-aware work-in-progress limit.
- 3 Workers run focused checks and submit a branch plus evidence manifest.
- 4 Verifiers reproduce the focused checks; unit managers integrate passing leaves into a wave branch.
- 5 Aggregate verification and independent review gate each wave.
- 6 Product verification runs continuously against integrated `main`, not only at the end.
- 7 Kai reconciles desired state from the requirement graph, repository, and gates, then opens the next waves or promotion gate.

The topology should expand when independent work is measurable and contract-stable, then contract when integration or release becomes the bottleneck. Unlimited token budget is useful only when it buys independent evidence or throughput; redundant context and duplicate work are still debt.

Framework Bug Ledger

Finding	Evidence from this run	Recommended Kensho change
Manager tail stall / insufficient self-refill	External overseer prompts were needed to refill independent work; upstream issue #354 records the 87/148 tail behavior.	Desired-state reconciler that dispatches safe work or records a concrete blocked reason.
Stale supervisor progress	Two prompts reported 87/148 after repository audits proved 148/148.	Generate status from canonical audits/job graph; reject regressive snapshots.
Direct jobs cannot bounce cleanly	<code>job bounce</code> could not requeue durable direct jobs without pipeline steps.	Make local direct delivery a first-class pipeline or preserve rejected branch/commit in direct-job retries.
Local-branch deliverable false negatives	Already merged branches appeared empty or missing against <code>main</code> ; Kai manually closed them with merge evidence.	Validate ancestry and recorded merge commit before diff-based failure.
Cleanup blocked by exited helpers	Several completed worktrees remained held as cwd by <code>SkyComputerUseClient</code> helpers; <code>lsof</code> handling also failed once.	Reap helper descendants and make no-open-file results non-errors with bounded retry.
External supervisor reply rejected	Kai could not reply to Kan because the daemon rejected an undeclared source instance.	Add a typed external-advisor/supervisor channel with explicit authority and reply routing.
Authority drift visible only in audit	36 authority-violation events were recorded.	Classify violations, summarize them in outcomes, and graduate stable rules from audit to enforcement.
Adaptive ceiling can reach zero	Host-load adjustment briefly queued all work with ceiling 0.	Add minimum-progress policy or explicit sustained-capacity escalation.
Declared pipeline bypass	188 direct jobs; zero pipeline jobs; no verifier jobs.	Report topology conformance and warn when specialized lanes are configured but unused.

Promotion Gates

Promotion is gate-based, not calendar-based.

Gate 1 - Product Safety

- Final row and column are reachable and editable.
- Unsaved work cannot be silently discarded.
- Required grid interactions are implemented or an explicit spec revision is approved.
- Native performance and accessibility evidence is recorded.

Gate 2 - Reproducible Acceptance

- All current core and frontend gates remain green.
- LibreOffice comparison is reproducible and deviations are machine-reconciled.
- Packaged-app workflows run automatically with retained evidence.
- CSP and dependency-security checks pass under an explicit policy.

Gate 3 - Release Candidate

- Candidate is built from a clean, tagged revision.
- Product identity, icon set, support matrix, documentation, and license posture are complete.
- Checksums, dependency inventory, and build provenance are generated by automation.

Gate 4 - Distribution Trust

- Developer ID signing passes strict verification.
- Notarization and stapling pass.

- Gatekeeper accepts the quarantined app and DMG in a clean environment.
- The exact candidate receives final native workflow and offline-network checks.

Only after all four gates should Excel Lite be described as a public v1 release.

Existing Supporting Evidence

- [Final UI and native product verification](#)
- [Native recalculation screenshot](#)
- [Packaged application screenshot](#)
- [Packaging and offline verification](#)
- [Conformance completeness audit](#)
- [Build-ready specification](#)

Audit Constraints

- The product checkout had no Git remote or tags, so hosted CI history and published-release state could not be examined.
- The job and daemon metrics describe recorded Kensho activity; runtime sums are not wall-clock duration and token totals include substantial cached input.
- The working project contained pre-existing orchestration state. The audited product revision itself was not modified during evidence collection.
- Native evidence is manual/accessibility-driven rather than a committed automated end-to-end suite.
- Advisory results are current as of the audit date and can change as databases evolve.
- No claim is made that untested formula semantics exactly match Microsoft Excel beyond the documented conformance contract and deviations.

Final Assessment

Excel Lite is a successful engineering prototype and a valuable Kensho experiment. The calculation engine is broad, tested, and demonstrably real; the native shell is usable; the application remains local; and the project generated enough objective evidence to support a controlled demonstration.

The product should not yet be distributed as a trusted public v1. The remaining blockers are concentrated in exactly the areas broad feature fan-out does not solve automatically: document safety, extreme-scale UI behavior, native automation, release provenance, and platform trust.

Kensho's strongest result was proving that a stable contract can support a large parallel implementation graph. Its weakest result was allowing the graph to collapse back into one manager at integration time. The next iteration should preserve the fan-out while recursively delegating domain integration, enforcing verifier and reviewer lanes, continuously testing the real packaged product, and computing progress from canonical state. That direction is more important than simply increasing the number of workers.